



Estrutura de Dados não Lineares

Prof. Thiago Koman Schmidt



Planejamento da aula

- Calendário acadêmico
- Plano de ensino
- Ferramentas usadas ao longo do semestre
- Algoritmos
- Análise de desempenho de algoritmos
- Tempo e espaço de execução



Plano de ensino

Plano de ensino

Conteúdo formativo:

- Algoritmos e complexidade;
- Ordem de Crescimento e Notação Assintóticas;
- Classes de algoritmos: logarítmico, linear, quadrático, exponencial e outros;
- Conceito e representação de árvores;
- Topologias de árvores: genéricas, binárias, AVL e B;
- Operações em árvores: percorrer, inserir e remover;
- Filas de prioridade;
- Conceitos de grafos: Ordem e grau;
- Multigrafo, completo e valorado;
- Representação de grafo;
- Matriz de adjacência, custo e incidência;
- Lista de adjacência e arestas;
- Árvores Geradoras Mínimas;
- Caminhos mínimos.

Plano de ensino - Avaliações

(10%) APS (prático):

- Script com métodos de busca

(20%) Avaliação parcial 1 (prático ? teórico):

- Algoritmos
- Árvores

(30%) Avaliação parcial 2 (prático ? teórico):

- Árvores
- Filas
- Grafos

(30%) Atividades em sala

(10%) PROCAD



Calendário Acadêmico

Calendário Acadêmico

Calendário Acadêmico Estudantes 2026/2

JULHO	D	S	T	Q	Q	S	S	23 a 23/07	Exame Final	5
				1	2	3	4	1	Registro de Notas Exame Final	
	5	6	7	8	9	10	11	10 a 12	Revisão de Férias Acadêmicas (estudantes)	
	12	13	14	15	16	17	18	16 a 16	Recesso Acadêmico (Férias dos professores)	
	19	20	21	22	23	24	25	17	Retorno dos Professores aos seus Campus	
	26	27	28	29	30	31		18 a 22	1ª Reunião Ordinária - NDE do Curso - 2026/2	
								23 a 25	Semana Pedagógica	
AGOSTO	D	S	T	Q	Q	S	S	27	Início 2º Semestre	24
							1	27 a 31	Open UNIVERSAL OFFICIAL INÍCIO! Nivelamento	
	2	3	4	5	6	7	8	02 a 07	1ª Reunião Ordinária - Colegiados de Curso - 2026/2	
	9	10	11	12	13	14	15	7	Secretaria Acadêmica - Fim do final para montagem de grade	
	16	17	18	19	20	21	22	14	Secretaria Acadêmica - Fim do final regularização de matrícula	
	23	24	25	26	27	28	29			
	30	31								
SETEMBRO	D	S	T	Q	Q	S	S	1	Período Independência do Brasil	23
				1	2	3	4	02	Período Novecentos dos Luz dos Fimões - Curitiba	
	6	7	8	9	10	11	12	11	Secretaria Acadêmica - Fim do final para cancelamento de disciplina repetida	
	13	14	15	16	17	18	19	21 a 25	Período de Avaliações 1	
	20	21	22	23	24	25	26	21 a 26	Somada de Aprendizagem: Apresentação Pré-Banca (1ª parcial)	
	27	28	29	30				22 a 24	2ª chamada de avaliação 1 + PROCAD (solução até 72 horas após a prova)	
								25/09 a 01/10	Avaliação de CPA	
OUTUBRO	D	S	T	Q	Q	S	S	10	Período Novecentos Apreendidos	23
					1	2	3	10	Recesso - Dia dos Professores	
	4	5	6	7	8	9	10	20/09 a 01/10	Avaliação de CPA	
	11	12	13	14	15	16	17	10	Secretaria Acadêmica - Fim do final para cancelamento de curso	
	18	19	20	21	22	23	24			
	25	26	27	28	29	30	31			
NOVEMBRO	D	S	T	Q	Q	S	S	1	Período Fimões	22
	1	2	3	4	5	6	7	10	Período Dia de Consciência Negra	
	8	9	10	11	12	13	14	21 a 27	Período de Avaliações 2	
	15	16	17	18	19	20	21	21 a 27	Somada de Aprendizagem: Apresentação Pré-Banca (2ª parcial)	
	22	23	24	25	26	27	28	28	Previsão de aplicação Enade - Engenharias	
	29	30						29/11 a 01/12	Somada de Aprendizagem: Apresentação Final para Indústria	
DEZEMBRO	D	S	T	Q	Q	S	S	01 a 02	2ª chamada de avaliação 2 + PROCAD (Solução até 72 horas após a prova)	4
							1	20/11 a 02/12	Somada de Aprendizagem: Apresentação Final para Indústria	
	6	7	8	9	10	11	12	3	Pré-entrevista para Concurso CONSU e CONCEPE 2026/2	
	13	14	15	16	17	18	19	4	Final 2º Semestre	
	20	21	22	23	24	25	26	10	Período Aniversário da Cidade "Londrina"	
	27	28	29	30	31			07 a 11	Exame Final	
								07 a 11	2ª Reunião Ordinária - NDE do Curso - 2026/2	
								11	Registro de Notas Exame Final	
								14	Fóris Acadêmicos (estudantes)	
								14 a 16	Atividades Acadêmicas (professores)	
								14 a 16	2ª Reunião Ordinária - Colegiados de Curso - 2026/2	
								21	Início das Férias Acadêmicas (professores)	



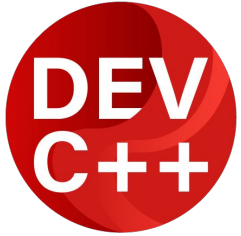
Linguagem e ferramentas

Linguagem de programação

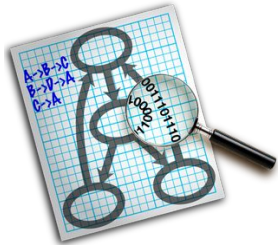


O **C++** é ideal por ser uma linguagem de alto desempenho, com forte controle de memória e suporte nativo a estruturas como **ponteiros**, **listas**, **filas**, **pilhas** e **árvores**, além de bibliotecas robustas para manipulação de **grafos**. Isso proporciona uma compreensão mais profunda e próxima da máquina sobre os algoritmos e estruturas abordadas, favorecendo o aprendizado dos conceitos fundamentais de complexidade e eficiência.

Ferramentas



<https://www.embarcadero.com/br/free-tools/dev-cpp>



<https://graphviz.org/>



<https://www.boost.org/>



Autoavaliação

Autoavaliação (1 a 5)

- Algoritmo e lógica de programação
- Variáveis locais e globais
- Tipos de variáveis (int, float, double, char, string, bool)
- Condicionais (If, else, else if e switch)
- Laço de repetição (while e for)
- Funções
- Passagem de parâmetros entre funções
- Funções recursivas
- Ponteiros
- Passagem de valor por referência
- Classe e Objeto
- Método e atributo
- Instância
- Abstração
- Encapsulamento
- Herança
- Polimorfismo

Autoavaliação (1 a 5)



<https://forms.gle/qYRcN5Pe61GkG8Zo7>



Algoritmos

Algoritmo

Um algoritmo é uma sequência finita de instruções bem definidas que resolvem um problema ou realizam uma tarefa.

Características:

- Entrada bem definida
- Saída esperada
- Clareza e não ambiguidade
- Efetividade (passos executáveis)
- Finiteza (termina em algum momento)

Exemplo: "Algoritmo para somar dois números"

1. Leia A
2. Leia B
3. Calcule $C = A + B$
4. Mostre C

Desafio (10 min)

Você foi contratado para desenvolver um algoritmo que descreva o processo de preparar café utilizando um coador e filtro de papel. O objetivo é transformar a sequência de ações em um algoritmo bem estruturado para que qualquer pessoa, mesmo sem conhecimento prévio, consiga seguir as instruções e obter café pronto para consumo.

O algoritmo deve obrigatoriamente:

- **Definir claramente a entrada** (ingredientes e utensílios necessários).
- **Definir a saída esperada** (resultado final do processo).
- **Ser claro e não ambíguo** (passos objetivos e ordenados).
- **Ser efetivo** (todos os passos devem ser executáveis no mundo real).
- **Garantir finitude** (o algoritmo deve terminar em algum momento).

Separe seu algoritmo em:

- Entrada
- Processo
- Saída

Desafio - Resultado

Entrada:

po_cafe $\leftarrow$ 3 colheres de sopa

agua $\leftarrow$ 500 ml

filtro $\leftarrow$ 1 unidade

coador $\leftarrow$ 1 unidade

recipiente $\leftarrow$ 1 unidade

Processo:

1. Colocar filtro dentro do coador
2. Adicionar po_cafe no filtro
3. Ferver agua
4. Despejar pequena quantidade de agua quente sobre o po_cafe para umedecer
5. Despejar o restante da agua quente de forma constante até completar 500 ml
6. Aguardar até que toda a agua passe pelo filtro
7. Retirar coador e descartar filtro com po_cafe usado
8. Servir o café no recipiente

Saída:

café_coado $\leftarrow$ café pronto para consumo

Por que Analisar Algoritmos?

O principal objetivo da análise de algoritmos é medir o quanto um algoritmo é eficiente, principalmente em dois aspectos:

1. **Tempo de execução:** quanto tempo ele leva para resolver um problema.
2. **Uso de memória (espaço):** quanto espaço ocupa em RAM durante a execução.

****** Mesmo que dois algoritmos resolvam o mesmo problema, um pode ser muito mais eficiente que o outro.

Espaço de Execução

Mede a quantidade de memória adicional necessária além da entrada.

Incluí:

- Variáveis temporárias
- Pilhas de chamadas (em recursão)
- Estruturas auxiliares (listas, vetores, dicionários)

******O tempo real de execução depende do hardware, mas a ordem de complexidade é o que importa para a análise.

Tempo de Execução

Mede o número de operações que um algoritmo executa em função do tamanho da entrada.

Tipos de análise:

- Pior caso (worst-case): cenário mais custoso
- Melhor caso (best-case): cenário ideal
- Caso médio (average-case): expectativa geral

****O tempo real de execução depende do hardware, mas a ordem de complexidade é o que importa para a análise.**

Ordem de complexidade

A **ordem de complexidade** descreve como o tempo ou o espaço de execução de um algoritmo cresce em relação ao tamanho da entrada n .

- $O(1)$: constante
- $O(n)$: linear
- $O(\log n)$: logarítmico
- $O(n^2)$: quadrática
- $O(2^n)$: exponencial
- $O(n \log n)$: linearítmica

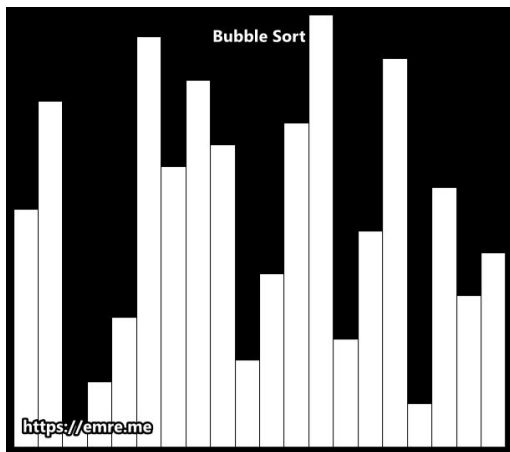
Comparação de Algoritmos

Problema: Ordenar uma lista de números

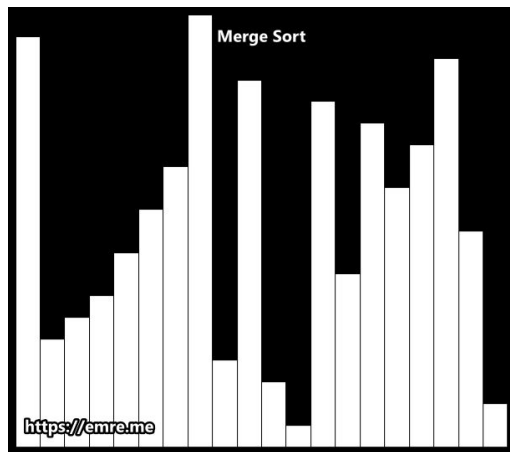
Entradas: [7, 12, 9, 2, 19, 3, 1, 18, 20, 5, 6, 4, 17, 11, 13, 8, 16, 14, 10, 15]

*random de 1 a 20

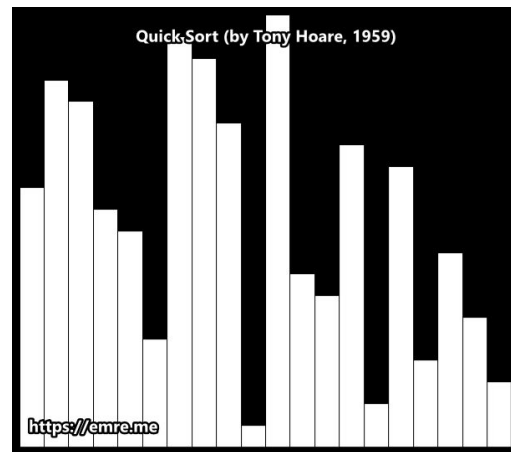
Algoritmos:



$O(n^2)$: quadrática



$O(n \log n)$: linearítmica



$O(n \log n)$: linearítmica

Fonte, algoritmos base e exemplo

<https://emre.me/algorithms/sorting-algorithms/>

https://colab.research.google.com/drive/1Z-YieLsu5n34YP8nMZt7VsCd_WGmjoy2?usp=sharing

Desafio

Implementar o algoritmo de **Bubble Sort** no DevC++, utilizando C++, e medir o tempo de execução.

Sugiro o uso na biblioteca `<ctime>` para medir o tempo.

DÚVIDAS?

